

Лабораторная работа № 2

Структуры данных в Julia

Цель работы

Изучить основные структуры данных языка Julia: кортежи, словари, множества, векторы, матрицы и многомерные массивы. Научиться создавать и преобразовывать массивы, выполнять операции над их элементами, применять фильтрацию, сортировку, статистические функции и пакет Primes.

Выполнение лабораторной работы

1. Множества и операции над ними

Для заданных множеств

```
A = Set{0, 3, 4, 9}
B = Set{1, 3, 4, 7}
C = Set{0, 1, 2, 4, 7, 8, 9}
P = union(intersect(A, B), intersect(A, C), intersect(B, C))
```

получено множество

```
P = Set{0, 1, 3, 4, 7, 9}
```

Таким образом, мощность множества P равна 6. Дополнительно были рассмотрены множества, содержащие элементы разных типов, и операция объединения множеств. На рисунке [-@fig-sets] показан пример работы с Set.

```
1
[10]: S1 = Set{1, "Julia", 2.44}
      S2 = Set{"Julia", :symbol, 1}

[10]: Set{Any} with 3 elements:
      :symbol
      "Julia"
      1

[11]: S3 = union(S1, S2)

[11]: Set{Any} with 4 elements:
      :symbol
      "Julia"
      1
      2.44
```

2. Создание и преобразование массивов

В начале работы был создан массив целых чисел:

```
N = 25
arr1 = collect(1:N)
```

Массив содержит числа от 1 до 25. С помощью функции `filter` были получены элементы, большие 20:

```
arr1_f = filter(x -> x > 20, arr1)
```

Результат равен [21, 22, 23, 24, 25]. Для обратного диапазона использовался массив `25:-1:1`.

Объединение прямого и обратного диапазонов дало массив, в котором числа от 1 до 25 повторяются в обратном порядке (рис. [-@fig-arrays]).

```
[51]: top_10_x = sort(x, rev = true)[1:10]
```

```
[51]: 10-element Vector{Int64}:  
      990  
      984  
      978  
      971  
      965  
      958  
      954  
      948  
      948  
      946
```

```
[52]: x_un = unique(x)
```

```
[52]: 215-element Vector{Int64}:  
      17  
     507  
     204  
     470  
     906  
     139  
     748  
      81  
     944  
     878  
     312  
     517  
     805  
      ⋮  
     971  
     888  
     665  
      25  
     407  
     920  
     965  
      43  
     610  
     248  
     222  
     128
```

```

[14]: arr1_f = filter(x -> x > 20, arr1)

[14]: 5-element Vector{Int64}:
      21
      22
      23
      24
      25

[15]: arr2 = collect(N:-1:1)
      arr2_f = filter(x -> x > 20, arr2)

[15]: 5-element Vector{Int64}:
      25
      24
      23
      22
      21

[16]: arr3 = vcat(collect(1:N), collect(N-1:-1:1))
      arr3_f = filter(x -> x > 20, arr3)

[16]: 9-element Vector{Int64}:
      21
      22
      23
      24
      25
      24
      23
      22
      21

```

Для формирования повторяющихся последовательностей применялись `fill`, `repeat` и `vcat`. Например:

```

tmp = [4, 6, 3]
arr5 = fill(tmp[1], 10)
arr6 = repeat(tmp, 10)
arr7 = vcat(fill(tmp[1], 11), fill(tmp[2], 10), fill(tmp[3], 10))
arr8 = vcat(fill(tmp[1], 10), fill(tmp[2], 20), fill(tmp[3], 30))

```

Размеры полученных векторов составили соответственно 10, 30, 31 и 60 элементов. Важное отличие состоит в том, что `repeat(tmp, 10)` повторяет весь вектор, а `fill(value, n)` повторяет один элемент (рис. [-@fig-repeat]).

```
[19]: arr6 = repeat(tmp, 10)
```

```
[19]: 30-element Vector{Int64}:
```

```
 4  
 6  
 3  
 4  
 6  
 3  
 4  
 6  
 3  
 4  
 6  
 3  
 4  
 ⋮  
 4  
 6  
 3  
 4  
 6  
 3  
 4  
 6  
 3  
 4  
 6  
 3
```

```
[21]: arr8 = vcat(fill(tmp[1], 10), fill(tmp[2], 20), fill(tmp[3], 30))
```

```
[21]: 60-element Vector{Int64}:
```

```
 4
 4
 4
 4
 4
 4
 4
 4
 4
 4
 4
 6
 6
 6
 ⋮
 3
 3
 3
 3
 3
 3
 3
 3
 3
 3
 3
 3
 3
 3
 3
```

3. Генераторы и векторные выражения

С помощью генераторов были сформированы следующие массивы:

```
M = factorial(2)
vec_12 = [2^i / i for i in 1:M]
vec_fn = ["fn$i" for i in 1:30]
```

Вектор `vec_fn` содержит строки от "fn1" до "fn30". Для получения пар значений использовался `zip`:

```
x_val, y_val = 0.1, 0.2
i_vals = 3:3:36
j_vals = 1:3:34
vec_pairs = collect(zip(x_val .^ i_vals, y_val .^ j_vals))
```

В результате получен вектор из 12 кортежей (рис. [-@fig-generators]).

```
(2.0000000000000000e+00, 2.0000000000000000e+00)
```

```
[33]: M = factorial(2)
      vec_12 = [2^i / i for i in 1:M]
```

```
[33]: 2-element Vector{Float64}:
      2.0
      2.0
```

```
[34]: vec_fn = ["fn$i" for i in 1:30]
```

```
[34]: 30-element Vector{String}:
      "fn1"
      "fn2"
      "fn3"
      "fn4"
      "fn5"
      "fn6"
      "fn7"
      "fn8"
      "fn9"
      "fn10"
      "fn11"
      "fn12"
      "fn13"
      ⋮
      "fn19"
      "fn20"
      "fn21"
      "fn22"
      "fn23"
      "fn24"
      "fn25"
      "fn26"
      "fn27"
      "fn28"
      "fn29"
      "fn30"
```

```
[30]: x_val, y_val = 0.1, 0.2
      i_vals = 3:3:36
      j_vals = 1:3:34
      vec_pairs = collect(zip(x_val.^i_vals, y_val.^j_vals))
```

```
[30]: 12-element Vector{Tuple{Float64, Float64}}:
      (0.0010000000000000002, 0.2)
      (1.0000000000000004e-6, 0.0016000000000000003)
      (1.0000000000000005e-9, 1.2800000000000006e-5)
      (1.0000000000000008e-12, 1.0240000000000006e-7)
      (1.0000000000000009e-15, 8.192000000000005e-10)
      (1.0000000000000008e-18, 6.5536000000000055e-12)
      (1.0000000000000012e-21, 5.2428800000000056e-14)
      (1.0000000000000012e-24, 4.194304000000005e-16)
      (1.0000000000000015e-27, 3.3554432000000044e-18)
      (1.0000000000000017e-30, 2.684354560000004e-20)
      (1.0000000000000018e-33, 2.147483648000004e-22)
      (1.000000000000002e-36, 1.7179869184000035e-24)
```

```
[33]: M = factorial(2)
      vec_12 = [2^i / i for i in 1:M]
```

```
[33]: 2-element Vector{Float64}:
      2.0
      2.0
```

Была также рассмотрена неоднозначная запись степеней. Для набора `tmp = [4, 6, 3]` выражение `2.^tmp` даёт `[16, 64, 8]`. При повторении каждого результата по четыре раза получается вектор, в котором число 64 встречается восемь раз.

```
[22]: arr9 = [2^tmp[1], 2^tmp[2], 2^tmp[3], 2^tmp[3], 2^tmp[3], 2^tmp[3]]
      count_6 = count(==(6), arr9)
```

```
[22]: 0
```

4. Операции над элементами массива

Для массива случайных целых чисел были выполнены поэлементные операции:

```
n = 250
x = rand(0:999, n)
y = rand(0:999, n)

vec_a = [y[i+1] - x[i] for i in 1:n-1]
vec_b = [x[i] + 2*x[i+1] - x[i+2] for i in 1:n-2]
vec_c = [sin(y[i]) / cos(x[i+1]) for i in 1:n-1]
```

Получены векторы размеров 249, 248 и 249 элементов. Примеры результатов вычислений представлены на рисунках `[-@fig-ops1]` и `[-@fig-ops2]`.

```
[39]: vec_b = [x[i] + 2*x[i+1] - x[i+2] for i in 1:n-2]
```

```
[39]: 248-element Vector{Int64}:
```

```
827
445
238
2143
436
1554
-34
1091
2388
985
541
1947
1043
⋮
2193
395
258
214
1282
2242
2138
84
849
1190
688
564
```

```
[37]: vrc_a = [y[i+1] - x[i] for i in 1:n-1]
```

```
[37]: 249-element Vector{Int64}:
```

```
577  
314  
-16  
246  
-740  
739  
-384  
17  
-98  
-279  
514  
-111  
-368  
⋮  
-483  
613  
650  
52  
-172  
-273  
-527  
523  
-235  
371  
549  
-213
```

```
[40]: vec_c = [sin(y[i]) / cos(x[i + 1]) for i in 1:n-1]
```

```
[40]: 249-element Vector{Float64}:
```

```
-2.44451031938799  
 0.24170866921901585  
-2.653407326755759  
-1.3887559569574826  
-0.389096905705355  
 0.5060349108028144  
-1.283888006094884  
-8.483902949464492  
 7.637355242916648  
 1.4240441342763455  
-4.186305419175218  
 0.3240546888313993  
 1.119960348152183  
  ⋮  
-0.20397379922149964  
-0.23324097994816426  
-1.5573602238294346  
-0.7729282699688248  
-0.3730267759499424  
 2.9162691159674834  
 1.352925705418898  
-0.7304841816390872  
 0.6368492865290384  
 0.9289492724672079  
 0.7834457579227202  
 1.186013047891023
```

Для определения среднего значения элементов равномерной сетки использовался пакет Statistics:

```
using Statistics  
x_vals = 3.0:0.1:6.0  
y_vals = [exp(x) * cos(x) for x in x_vals]  
mean_y = mean(y_vals)
```

Получено значение $\text{mean}_y \approx 53.11374594642971$ (рис. [-@fig-mean]).

```
[23]: x_vals = 3.0:0.1:6.0
```

```
[23]: 3.0:0.1:6.0
```

```
[25]: using Statistics
```

```
[27]: y_vals = [exp(x) * cos(x) for x in x_vals]  
      mean_y = mean(y_vals)
```

```
[27]: 53.11374594642971
```

```
[28]: y_vals = [exp(x) * cos(x) for x in x_vals]  
      mean_y = mean(y_vals)
```

```
[28]: 53.11374594642971
```

Для каждого элемента массива `x` также вычислялось отклонение от среднего:

```
x_mean = mean(x)  
vec_dev = [abs(v - x_mean)^(1/2) for v in x]
```

Результат показан на рисунке [-@fig-deviation].

```
[44]: x_mean = mean(x)
      vec_dev = [abs(val - x_mean)^(1/2) for val in x]
```

```
[44]: 250-element Vector{Float64}:
      21.78522435046286
      3.9247929881714776
      16.958655607093387
      4.64714966404139
      20.35691528694856
      18.777539774954544
      16.012620022969383
      20.263168557755225
      21.26979078411445
      19.657161544841614
      13.401343216260077
      5.040238089614418
      17.703220046082013
      ⋮
      21.60083331725885
      13.099465637956381
      9.197608384792211
      20.697922601072793
      21.75784915840718
      10.789068541815832
      21.18008498566519
      10.881360209091508
      8.808859176987676
      15.607562269617892
      16.419378794582943
      19.06819341206712
```

```
[45]: y_max = maximum(y)
      to_max = count(val -> (y_max - val) <= 200, y)
```

```
[45]: 39
```

5. Фильтрация, подсчёт и сортировка

Для анализа массивов `x` и `y` применялись `findall`, `count`, `maximum`, `sort` и `sortperm`.

Например:

```
idx_gt_600 = findall(>(600), y)
y_gt_600 = y[idx_gt_600]
y_max = maximum(y)
to_max = count(v -> y_max - v <= 200, y)
even_count = count(iseven, y)
odd_count = count(isodd, y)
ycount_7 = count(v -> v % 7 == 0, x)
```

На скриншоте показано, что для конкретной последовательности случайных чисел число элементов `y`, больших 600, равно 83. Число чётных элементов `y` равно 137, нечётных — 113, а количество элементов `x`, кратных 7, равно 34 (рис. [-@fig-count]). Эти значения зависят от последовательности случайных чисел, если перед генерацией не установить `Random.seed!`.

```
[46]: even_count = count(iseven, y)
      odd_count = count(isodd, y)

[46]: 113

[47]: even_count = count(iseven, y)

[47]: 137

[48]: odd_count = count(isodd, y)

[48]: 113

[49]: ycount_7 = count(val -> val % 7 == 0, x)

[49]: 34
```

Сортировка выполнялась двумя способами:

```
x_sort = x[sortperm(y)]
top_10_x = sort(x, rev=true)[1:10]
```

В первом случае элементы x упорядочиваются по значениям соответствующих элементов y , во втором — выбираются десять наибольших элементов самого массива x (рис. [-@fig-sort]).

```
[43]: x_corr = x[idx_gt_600]
```

```
[43]: 83-element Vector{Int64}:
```

```
 17
204
906
748
878
517
122
483
678
801
391
 42
358
 ⋮
199
 18
431
356
626
441
320
407
965
608
248
222
```

Для удаления повторений использовалась функция `unique(x)`. Она возвращает массив, содержащий только различные значения исходного массива.

```
[50]: x_sort = x[sortperm(y)]
```

```
[50]: 250-element Vector{Int64}:
```

```
206
```

```
128
```

```
201
```

```
74
```

```
361
```

```
144
```

```
449
```

```
267
```

```
351
```

```
837
```

```
482
```

```
971
```

```
745
```

```
⋮
```

```
359
```

```
64
```

```
213
```

```
391
```

```
407
```

```
581
```

```
414
```

```
626
```

```
309
```

```
627
```

```
431
```

```
241
```

6. Простые числа и квадраты чисел

Для построения массива квадратов первых ста натуральных чисел использовалось выражение:

```
squares = [i^2 for i in 1:100]
```

Массив содержит 100 элементов; последние значения равны 9604, 9801 и 10000 (рис. [-@fig-squares]).

```
[53]: squares = [i^2 for i in 1:100]
```

```
[53]: 100-element Vector{Int64}:
```

```
 1
 4
 9
16
25
36
49
64
81
100
121
144
169
 ⋮
7921
8100
8281
8464
8649
8836
9025
9216
9409
9604
9801
10000
```

Для работы с простыми числами был подключён пакет Primes:

```
using Pkg
Pkg.add("Primes")
using Primes

primes1 = primes(1000)
prime89 = primes1[89]
primes_slice = primes1[89:99]
```

В диапазоне от 1 до 1000 найдено 168 простых чисел. 89-е простое число равно 461. Срез с 89-го по 99-е простое число имеет вид:

```
[461, 463, 467, 479, 487, 491, 499, 503, 509, 521, 523]
```

Результаты работы с пакетом Primes представлены на рисунке [-@fig-primes].

```
[56]: using Pkg
      Pkg.add("Primes")
      using Primes

      Resolving package versions...
      Installed IntegerMathUtils - v0.1.4
      Installed Primes - v0.5.7
      Updating `C:\Users\Кирилл\.julia\environments\v1.12\Project.toml`
      [27ebfcd6] + Primes v0.5.7
      Updating `C:\Users\Кирилл\.julia\environments\v1.12\Manifest.toml`
      [18e54dd8] + IntegerMathUtils v0.1.4
      [27ebfcd6] + Primes v0.5.7
      Precompiling packages...
      2160.9 ms ✓ IntegerMathUtils
      1002.5 ms ✓ Primes
      2 dependencies successfully precompiled in 5 seconds. 40 already precompiled.

[57]: primes1 = primes(1000)[1:168]
      prime89 = primes1[89]
      primes_slice = primes1[89:99]

[57]: 11-element Vector{Int64}:
      461
      463
      467
      479
      487
      491
      499
      503
      509
      521
      523
```

7. Вычисление сумм

Были вычислены три суммы:

```
sum_6_1 = sum(i^3 + 4*i^2 for i in 10:1000)
M_6 = 25
sum_6_2 = sum((2^i)/i + (3^i)/i for i in 1:M_6)
```

Для третьей суммы использовалось накопление в типе Float64, чтобы избежать переполнения целого типа при вычислении произведения:

```
total_sum = 1.0
cur_p = 1.0
for i in 1:19
    cur_p *= (2*i) / (2*i + 1)
    total_sum += cur_p
end
```

Полученные значения:

```
sum_6_1 = 251835580835
sum_6_2 ≈ 5.1948766717837875e10
sum_6_3 ≈ 6.976346137897618
```

Примеры ячеек с вычислением сумм приведены на рисунке [-@fig-sums].

```
[59]: sum_6_1 = sum([i^3 + 4*i^2 for i in 10:1000])

[59]: 251835580835

[60]: M_6 = 25
      sum_6_2 = sum([(2^i) / i + (3^i) / i for i in 1:M_6])

[60]: 5.1948766717837875e10

[61]: total_sum = 1.0
      cur_p = 1.0
      for i in 1:19
          cur_p *= (2^i) / (2^i + 1)
          total_sum += cur_p
      end
      sum_6_3 = total_sum

[61]: 6.976346137897618

[ ]: 0
```

Выводы

В ходе лабораторной работы были изучены основные структуры данных Julia и операции над ними. Были рассмотрены множества и операции пересечения и объединения, создание массивов с помощью диапазонов, fill, repeat, vcat и генераторов. Выполнены поэлементные арифметические и тригонометрические преобразования массивов.

С помощью функций filter, findall, count, unique, sort и sortperm выполнены фильтрация, подсчёт, удаление повторений и сортировка элементов. Подключение пакета Primes позволило найти 168 простых чисел до 1000 и определить 89-е простое число — 461. Также были вычислены квадраты чисел от 1 до 100 и три контрольные суммы.

При выполнении работы были учтены особенности Julia: индексация начинается с единицы, repeat и fill имеют разное назначение, а вычисление произведений больших целых чисел может привести к переполнению. Для воспроизводимости результатов генерации случайных массивов рекомендуется использовать Random.seed!.